

SEAGAS — Sprint 2 Daily Work Report

Carl (Soh Way Miin) — Backend Developer | 30 September 2026
COS40005 Computing Technology Project A | Group 10 | POC004

Today's Work Summary

Task	Status	Sprint Ticket
Verify jobs.py sort order bug	Confirmed Fixed	S1 carryover
Weighted readiness scoring (required=3, preferred=2, bonus=1) completed	Completed	S2-05
Merge recommendations library v1+v2+v3 into one final file	Completed	S2-03
Remove 174 duplicate resources from database	Completed	S2-03
Add UNIQUE constraint on recommendations_library	Completed	S2-03
Fix SQL file encoding (WIN1252 -> UTF-8, remove BOM)	Completed	S2-03
PR #7 pushed (features/recommendation_engine -> dev)	Awaiting review	S2-03, S2-04
Commit and push all Sprint 2 backend work	Completed	S2-03 to S2-05

Task 1 — Verify jobs.py Sort Order Bug

What was the issue?

Sprint 1 recorded a known bug: job_role_skills were sorting alphabetically by importance (bonus before preferred before required), instead of required first. This was flagged as a carryover task for Sprint 2.

What was done?

Reviewed the current jobs.py code. The GET /roles/{role_id}/skills endpoint already contained a CASE WHEN sort expression. Ran a verification query against the DATA_ANALYST role to confirm the ordering was correct.

Verification query run:

```
SELECT sl.skill_name, jrs.importance FROM job_role_skills jrs JOIN skills_library sl ON
sl.skill_id = jrs.skill_id WHERE jrs.role_id = (SELECT role_id FROM job_roles WHERE
role_code = 'DATA_ANALYST' LIMIT 1) ORDER BY CASE jrs.importance WHEN 'required' THEN 1
WHEN 'preferred' THEN 2 WHEN 'bonus' THEN 3 END LIMIT 15;
```

Result:

Check	Result
First 9 rows	All required (Python, SQL, Excel, Pandas, Power BI, Data Analysis, Data Clean
Rows 10-15	preferred (Attention to Detail, Communication, Critical Thinking, ETL, Presentat
Sort order	required -> preferred -> bonus CONFIRMED CORRECT
Bug status	Already fixed in previous session — no change needed

Task 2 — Weighted Readiness Scoring (assessments.py)

What was the problem?

The original readiness score formula in assessments.py treated all skills equally regardless of importance level. Missing a 'bonus' skill and missing a 'required' skill both had the same penalty, which produced misleading scores.

Old formula:

```
readiness_score = (strong_count * 1.0 + developing_count * 0.5) / total * 100
```

What was changed?

Rewrote the matching loop and scoring logic in assessments.py to use importance-weighted points. Each skill now contributes a different number of points based on its importance level. The dimension scores (Technical, AI/Digital, Analytical, Soft) were also updated to use the same weighted formula within each category.

New scoring weights:

Importance Level	Weight (max points)	Strong match earns	Developing earns	Gap earns
required	3 pts	3.0 pts (100%)	1.5 pts (50%)	0 pts
preferred	2 pts	2.0 pts (100%)	1.0 pts (50%)	0 pts
bonus	1 pt	1.0 pt (100%)	0.5 pts (50%)	0 pts

New formula:

```
readiness_score = earned_points / max_points * 100
```

Test result (POST /api/assessments/run, role: Data Analyst, student0001):

Field	Value	Meaning
readiness_score	45%	Weighted score for this student vs Data Analyst role
scoring_method	weighted (required=3, preferred=2, bonus=1)	Confirmed new logic applied
earned_points	22.5	Total points earned by student
max_points	50	Maximum possible points for this role
strong	8	Skills with exact match
developing	6	Skills with partial match
gaps	10	Skills missing entirely
total skills	24	Total skills required for Data Analyst
Calculation check	22.5 / 50 x 100 = 45%	CORRECT

Task 3 — Recommendations Library Cleanup and Merge

What was the problem?

Three separate SQL seed files existed: seed_recommendations_library.sql (v1), seed_recommendations_library_v2.sql, and seed_recommendations_library_v3.sql. Running all three files introduced duplicate resources because some skills appeared in multiple files. The database contained 20+ duplicate entries visible on the Recommendations page (e.g. 'A/B Testing by Google' appeared twice).

Step-by-step actions taken:

Step	Action	Command / Method
------	--------	------------------

1	Diagnosed duplicates in database	SELECT resource_title, skill_id, COUNT(*) FROM recommendations_library GROUP BY resource_title, skill_id
2	Found 20 duplicate resource-skill pairs (all counts = 2) every result confirmed	
3	Deleted duplicates using ctid comparison	DELETE FROM recommendations_library a USING recommendations_library b WHERE a.resource_title = b.resource_title AND a.skill_id = b.skill_id AND a.ctid > b.ctid
4	Verified count after deduplication	SELECT COUNT(*) -> 298 rows
5	Merged all 3 SQL files into seed_recommendations_library_final.sql sections labeled v1/v2/v3	
6	Deleted old v1, v2, v3 files	del database\seed_recommendations_library*.sql (kept final only)
7	Fixed file encoding (special characters causing Win1252/UTF8 error) encoding.py (replaced em dashes, smart quotes, removed 0x90/0x9D bytes)	
8	Fixed BOM issue (ï»¿ prefix after PowerShell UTF8 save) fix_encoding.py to use utf-8-sig for reading	
9	Re-ran final SQL file (ON CONFLICT DO NOTHING) -f database\seed_recommendations_library_final.sql	
10	Added UNIQUE constraint to prevent future duplicates	ALTER TABLE recommendations_library ADD CONSTRAINT uq_resource_skill UNIQUE (skill_id, resource_title)
11	Final verification: 0 skills without resources, 0 duplicate resource-skill pairs, 326 total queries all passed	

Final database state:

Metric	Value	Status
Total resources in recommendations_library	326	PASS
Skills without any resource	0 of 138	PASS
Duplicate resource-skill pairs	0	PASS
UNIQUE constraint active	uq_resource_skill (skill_id, resource_title)	PASS
Recommendations page rendering	Working (A/B Testing, Attention to Detail, etc)	PASS

Errors Encountered and How They Were Resolved

Error	Cause	Resolution
function min(uuid) does not exist	UUID columns cannot use MIN() in PostgreSQL	Used ctid comparison method instead: DELETE ... USING ... WHERE
role_code = DA returned 0 rows	role_code in database is DATA_ANALYST	Changed to WHERE role_code = DATA_ANALYST
character 0x90/0x9D in WIN1252 has SQL file obtained UTF8	SQL file obtained UTF8	Created fix_encoding.py (replaced special chars and save as plain UTF-8)
SyntaxError: unterminated string literal	Tried to run multi-line Python code in PowerShell	Moved directly to .py file and ran with python backend/fix_encoding.py
ï»¿ syntax error at start of SQL	PowerShell Set-Content -Encoding UTF8 added BOM	Used BOM file encoding mark to read with utf-8-sig to strip BOM, write with utf-8-sig
INSERT 0 298 but total became 596	ON CONFLICT DO NOTHING requires UNIQUE	Re-ran and DELETE to delete duplicates, then added UNIQUE constraint

Git Commits Made Today

Commit Hash	Message	Files Changed
76bb807	complete recommendations library (472 resources, 138 skills)	database/seed_recommendations_library_v2.sql, database/seed_recommendations_library_v1.sql, database/seed_recommendations_library_v3.sql
0cc533e	feat: weighted readiness scoring (required=3, preferred=2, max=5)	backend/assessments.py
e94ecb1	git add database/seed_recommendations_library_final.sql	database/seed_recommendations_library_final.sql
99d675f	feat: weighted scoring + merged recommendations library (326 resources, 0 duplicates)	database/seed_recommendations_library_final.sql, backend/assessments.py

Sprint 2 Backend Task Status (Carl)

Task	Sprint Ticket	Status	Blocker
jobs.py sort order bug	S1 carryover	Done	None
Weighted readiness scoring	S2-05	Done	None
Recommendations library (326 resources, 0 duplicates)	S2-05	Done	None
UNIQUE constraint on recommendations_library	S2-03	Done	None
PR #7 merge to dev	S2-03, S2-04	Awaiting review	Waiting for renhang0731 + yonghin approve
NLP module API integration	S2-06	Not started	Waiting for Yong Hin NLP module
schema.sql UNIQUE constraint update	S2-03	Pending	Minor — can do anytime